LX Notes ↔ TheCuelist Integration

Nick Solyom — LX Notes team

2026-05-13

CONTENTS

LX Notes ↔ TheCuelist Integration — Technical Proposal

Author: LX Notes team **Audience:** TheCuelist.com engineering **Status:** Proposal — comments and counter-proposals welcome **Date:** 2026-05-13

---

1. Summary

We'd like to integrate LX Notes (`lxnotes.app`) with TheCuelist (`thecuelist.com`) so that a lighting designer, stage manager, or director can move seamlessly between the script-with-cues view in TheCuelist and the notes-about-cues view in LX Notes. The integration is built around four user-facing journeys, all flowing through one stable pairing: **one LX Notes production ↔ one TheCuelist show**.

What we're asking from TheCuelist:

- A small read-only HTTP API (one endpoint) that exposes a show's script structure: pages, acts, scenes/songs, and cues.
- An outbound webhook fired when that structure changes.
- A "Create LX Note" deep-link button on the cue list / script viewer that opens our Add-Note dialog with the cue pre-filled.

What LX Notes provides in return:

- A webhook receiver for the structure-changed signal.
- An "Open in TheCuelist" deep link on every cue note, jumping to the right cue on your script viewer.
- A stable shared HMAC secret per linked production so all server-to-server calls can be cheaply verified.

No new user accounts, no SSO, no token exchange. Accounts in each app stay separate; the link is at the show/production level.

---

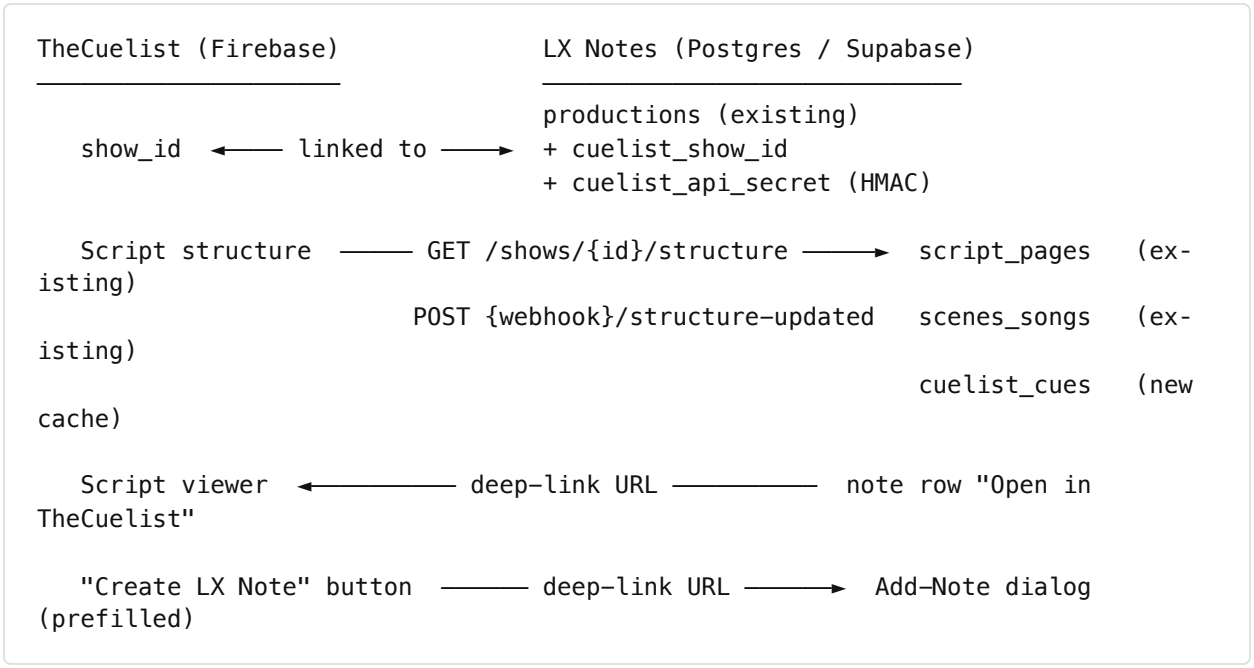
2. The four features

#	Journey	App A → App B
1	Script structure flows into LX Notes so notes can be filtered, grouped, and labeled by act / scene / song / page	TheCuelist → LX Notes (data sync)
2	When a user writes “Cue 47 is too slow” in LX Notes, the page number, act, and scene/song auto-fill from the synced structure	LX Notes internal, powered by the data from #1
3	A user reading the script in TheCuelist clicks a cue → lands in LX Notes’ Add-Note dialog with cue + script context pre-filled	TheCuelist → LX Notes (deep link)
4	A user reviewing notes in LX Notes clicks a note → lands on the right page of TheCuelist’s script viewer	LX Notes → TheCuelist (deep link)

Features #1 and #2 require the API. Features #3 and #4 are URL conventions only — no live calls needed at click time.

---

3. Architecture overview



Key insight: LX Notes already has the receiving tables. `script_pages` (`page_number`, `act_name`, `act_first_cue_number`, `first_cue_number`), `scenes_songs` (`name`, `type`, `order_index`, `act_id`, `script_page_id`, `continues_from_id`), and FK columns on `notes`

(`script_page_id`, `scene_song_id`) have been in the schema since early 2026. We also have two atomic bulk-replace RPCs ready to receive the sync (`replace_script_pages`, `replace_scenes_songs`). We're plugging into existing infrastructure, not designing new tables on our side.

---

4. API contract

This is the load-bearing part of the proposal. Everything below is a starting point; we expect to iterate together.

4.1 `GET /API/V1/SHOWS/{SHOW_ID}/STRUCTURE`

Returns the full script structure for one show. Called by LX Notes:

- once on initial link,
- on demand when a user clicks "Re-sync from TheCuelist," and
- whenever the structure-updated webhook fires.

Auth: `X-Integration-Signature` + `X-Integration-Timestamp` headers (see §5). On a GET with no body, sign the **canonical URL** = path + `?` + sorted-query-string (e.g. `/api/v1/shows/abc123/structure?foo=1&bar=2` → sign `/api/v1/shows/abc123/structure?bar=2&foo=1`). If no query string, sign the path alone. Keeping the format strict avoids silent signature failures from inconsistent query normalization on either side.

Caching: every response includes an `ETag` header. LX Notes sends `If-None-Match: "<etag>"` on subsequent pulls; return `304 Not Modified` (empty body, signed headers only) if unchanged. This makes "re-sync on every webhook" cheap during tech week, when the structure may change dozens of times in an hour.

Response (proposed shape — we are flexible):

```
{
  "show_id": "abc123",
  "show_name": "Hamilton",
  "updated_at": "2026-05-13T12:00:00Z",
  "pages": [
    {
      "id": "page_001",
      "page_number": "1",
      "act_name": "Act I",
      "first_cue_id": "cue_001"
    }
  ],
  "acts": [
    { "id": "act_1", "name": "Act I", "order_index": 0 }
  ],
  "scenes_songs": [
    {
      "id": "ss_001",
      "type": "song",
      "name": "Alexander Hamilton",
      "act_id": "act_1",
      "order_index": 0,
      "start_page_id": "page_001",
      "continues_from_id": null
    },
    {
      "id": "ss_002",
      "type": "scene",
      "name": "Aaron Burr's Office",
      "act_id": "act_1",
      "order_index": 1,
      "start_page_id": "page_004",
      "continues_from_id": null
    }
  ],
  "cues": [
    {
      "id": "cue_001",
      "label": "Cue 1",
      "page_id": "page_001",
      "scene_song_id": "ss_001",
      "order_index": 0
    }
  ]
}
```

Field-by-field mapping into LX Notes' tables:

Response field	Lands in	Column / note
`pages[].id`	`script_pages`	`cuelist_id` (new column we'll add — TheCuelist's string ID, indexed UNIQUE per production). Internal UUID `id` stays. Sync matches by `cuelist_id`.
`pages[].page_number`	`script_pages`	`page_number` (text — we already store strings like `"42a"`)
`pages[].act_name`	`script_pages`	`act_name`
`pages[].first_cue_id`	`script_pages`	`act_first_cue_number`, `first_cue_number` — **transform during sync:** look up the cue in the `cues[]` array, take its `label`, store the label string.
`scenes_songs[].id`	`scenes_songs`	`cuelist_id` (new column we'll add — same pattern as pages)
`scenes_songs[].name`	`scenes_songs`	`name`
`scenes_songs[].type`	`scenes_songs`	`type` (`"scene"` or `"song"`)
`scenes_songs[].order_index`	`scenes_songs`	`order_index`
`scenes_songs[].act_id`	`scenes_songs`	`act_id` (we map TheCuelist's act_id to one of our internal act rows; stored as text mirroring your ID)
`scenes_songs[].start_page_id`	`scenes_songs`	`script_page_id` — resolved via `cuelist_id` lookup
`scenes_songs[].continues_from_id`	`scenes_songs`	`continues_from_id` — resolved the same way
`cues[]` (whole array)	`cuelist_cues` (new)	Cache. `cuelist_cue_id` is the join key; not stored on `notes`.

ID stability is load-bearing. Sync matches rows by your `cuelist_id`, not by row position. If your IDs are stable across script edits, our existing `script_pages.id` / `scenes_songs.id` UUIDs (which `notes` FK into) survive renumbering and renames. If your IDs change every edit,

every sync would replace every row, NULL out every cue note's `script_page_id` / `scene_song_id` (the existing FK is `ON DELETE SET NULL`), and silently lose script context on every note. See open question #2 in §8.

Errors:

HTTP	Meaning	LX Notes behavior
`200`	OK	Apply sync
`401`	Bad / missing signature	Surface "Re-link to TheCuelist" in our settings UI
`404`	Show doesn't exist or isn't linked	Surface "This production is no longer linked"
`429`	Rate-limited	Exponential backoff, retry up to 3×
`5xx`	Server error	Retry once, then surface a soft-fail banner

Pagination / size: for v1, return the whole structure in one response. Hamilton-sized shows are well under a megabyte; we don't need streaming. If you need pagination later, we can add it.

4.2 `POST {LXNOTES_WEBHOOK_URL}/STRUCTURE-UPDATED` (THECUELIST → LX NOTES)

Fired when anything in the structure changes (page added, cue inserted, scene renamed). LX Notes gives you the URL at link time.

Request body:

```
{
  "show_id": "abc123",
  "updated_at": "2026-05-13T12:00:00Z"
}
```

Auth: same HMAC scheme as §4.1 (sign the body).

LX Notes response contract:

- Returns `200` within ~500ms regardless of outcome.
- The pull happens asynchronously after the 200.
- If the signature is bad, returns `401` and does nothing.

Retry expectations: if you don't get a `200`, retry with exponential backoff for up to 24 hours.

Async pull failure handling (our side): because LX Notes returns 200 before the pull runs, we can't propagate pull failures back through the webhook. Our side will: (a) retry the pull up to 3× with backoff, (b) record `productions.cuelist_last_sync_error` and `cuelist_last_sync_at` so admins see staleness in the settings UI, and (c) surface a banner on the production's notes page when the cache is more than 24h stale or last sync errored. Users can then click "Re-sync from TheCuelist" to force a retry.

4.3 DEEP-LINK URLS (BOTH DIRECTIONS)

These are **URL conventions**, not API calls. No auth needed — the user's session in the destination app handles permissions.

LX Notes → TheCuelist (Feature 4)

We propose (open to your alternative):

```
https://thecuelist.com/show/{show_id}/script?cue={cue_id}
```

If `cue`-precision isn't feasible, a `page` fallback is also fine:

```
https://thecuelist.com/show/{show_id}/script?page={page_number}
```

LX Notes will use whichever you support and fall back to `page` if `cue` returns "not found" on your side.

TheCuelist → LX Notes (Feature 3)

LX Notes commits to this URL:

```
https://www.lxnotes.app/notes?action=new&module=cue&show_id=
{show_id}&cuelist_cue_id={cue_id}&cue_label={label}
```

- `cuelist_cue_id` and `cue_label` are both useful. We prefer `cuelist_cue_id` for precision; `cue_label` is a fallback if a freshly added cue hasn't synced yet.
- LX Notes' notes page detects the `action=new&module=cue` combination, resolves cue → page/scene/song from our cache, and opens the Add-Note dialog with everything pre-filled. The user confirms by clicking Save.
- If the user isn't signed in, our normal Google OAuth flow runs, then the dialog opens.

---

5. Auth — HMAC per-production secret

We deliberately avoid OAuth between users. The trust boundary is the *production/show* link, not the individual user.

HOW IT WORKS

1. At link time, **LX Notes generates a 32-byte cryptographically-random secret**. This is shown once in the LX Notes UI and POSTed to a TheCuelist endpoint (see §6 for the linking flow). Both apps store it server-side keyed by the linked production/show.

2. **Every cross-app HTTP request** includes:

   ```
   X-Integration-Timestamp: 1715616000
   X-Integration-Signature: sha256=<hex digest>
   ```

   where the signature is:

   ```
   HMAC-SHA256( "{timestamp}.{request_body_or_path}", secret )
   ```

   - For requests *with a body* (POST webhook), sign `timestamp + "." + raw_body_bytes`.
   - For requests *without a body* (GET), sign `timestamp + "." + request_path`.

3. **Receivers reject** any request where:

   - Timestamp is more than 5 minutes from server time, OR
   - Signature comparison (constant-time) fails.

   Replay protection: within the 5-minute window, a captured signed request *could* be replayed. We accept this risk — it matches the Stripe / GitHub webhook model. The webhook payload itself (`{ show_id, updated_at }`) is idempotent: replaying it just re-pulls the same structure. Mutating endpoints (link / unlink) are admin-rare and behind the 5-min window; if you'd prefer a nonce, we can add `X-Integration-Nonce` and dedupe on our side.

PSEUDOCODE (NODE — ADAPT TO YOUR STACK)

Signing:

```
import crypto from 'node:crypto';

function sign(secret, timestamp, payload) {
  const message = `${timestamp}.${payload}`;
  return 'sha256=' + crypto
    .createHmac('sha256', secret)
    .update(message)
    .digest('hex');
}
```

Verifying:

```
function verify(secret, headers, payload) {
  const ts = parseInt(headers['x-integration-timestamp'], 10);
  const sig = headers['x-integration-signature'];
  if (!ts || !sig) return false;
  if (Math.abs(Date.now() / 1000 - ts) > 300) return false; // 5 min
  const expected = sign(secret, ts, payload);
  return crypto.timingSafeEqual(
    Buffer.from(sig),
    Buffer.from(expected)
  );
}
```

ROTATION

Either side can hit “Rotate secret” in their settings UI. Old secret is accepted for 60 minutes after rotation so a webhook in flight doesn’t get rejected.

RATE LIMITING

Recommend a per-show rate limit on the structure endpoint (~30 requests/minute is more than enough; structure changes are rare). LX Notes will request thoughtfully — never more than once per visible-to-user action plus webhook-triggered pulls.

---

6. Linking flow

One-time setup, performed by an admin on each side:

1. **LX Notes user** opens *Settings → Link to TheCuelist*. They paste their TheCuelist show ID (we can iterate later to a friendlier handoff, e.g. an OAuth-less “Authorize on TheCuelist” button that returns the ID).

2. **LX Notes generates** a 32-byte HMAC secret. Displays it once in the UI for safekeeping.
3. **LX Notes POSTs** to a TheCuelist endpoint (we propose `POST /api/v1/integrations/lxnotes/link`; final path is yours to pick):

   ```
   {
     "lxnotes_production_id": "uuid-here",
     "lxnotes_short_code": "ABC123",
     "lxnotes_show_id_claim": "abc123",
     "webhook_url": "https://www.lxnotes.app/api/cuelist/webhooks/structure-up-
   dated",
     "secret": "hex-encoded-32-bytes"
   }
   ```

   Authed with the secret itself (signature header verifying the body).

   Webhook URL must be the stable production domain — `https://www.lxnotes.app/...`. LX Notes runs on Vercel, where preview deploys get ephemeral URLs (`lx-notes-git-*.vercel.app`). If TheCuelist captures one of those, the link breaks on the next deploy. We will reject any link request whose `webhook_url` doesn't match an allow-listed production host.
4. **TheCuelist responds** `200` once the secret is stored on the show. Returns `409` if the show is already linked to a different LX Notes production.
5. **LX Notes immediately calls** `GET /api/v1/shows/{show_id}/structure` to seed the cache. If it succeeds, the link is confirmed in our UI.

Unlinking: either side can sever. We'll POST `DELETE /api/v1/integrations/lxnotes/link` (signed) to notify you when unlinking from our side. We expect a symmetric notification from your side.

---

7. Data model decisions LX Notes has pre-made

These are settled on our side. Sharing here so you know what we can flex on and what we can't.

Decision	Rationale
Cue identity stored on `cuelist_cues` (cache) only, **not on `notes`**	Notes hold `cue_number` as a free-text string. The cache resolves label → cuelist_cue_id when we need to build a deep link. Allows notes to exist before linking and survive cue renumbering.
`notes.script_page_id` and `notes.scene_song_id` are **the long-term source of truth** for "where in the script this note belongs"	Even if cue labels renumber, the position in the script stays correct.
No SSO between user accounts	Linking is at the show/production level. Each user signs into each app on their own.
HMAC, not OAuth, for server-to-server	Simpler, cheaper, scoped per-show, easy to rotate.
Sync writes use **new upsert RPCs**: `upsert_script_pages_by_cuelist_id` and `upsert_scenes_songs_by_cuelist_id`. INSERT … ON CONFLICT (production_id, cuelist_id) DO UPDATE.	Preserves existing UUIDs so `notes.script_page_id` / `notes.scene_song_id` FK references survive every sync. The older `replace_*` RPCs stay for the manual script-entry admin flow — two callers, two patterns, no conflation.
`productions.short_code` (existing 6-char shareable code) is available as your join key if you'd prefer it over our UUID	Lighter to display in your UI; we don't mind which one you store.
`productions.cuelist_show_id` is **UNIQUE** across all productions	One CueList show can be linked to at most one LX Notes production. Prevents ambiguous routing on inbound deep links (`?show_id=` → which production?). If a co-production needs to share script data across multiple LX Notes productions, we'll revisit with you.
`script_pages` and `scenes_songs` will get a `cuelist_id` text column, indexed `UNIQUE (production_id, cuelist_id)`	Lets the sync match rows by TheCuelist's stable ID rather than regenerating our UUIDs. Without this, every sync would re-create rows and orphan existing FK references on `notes`.

8. Open questions for TheCuelist team

Each of these is a checkpoint where your answer drives our spec.

1. `show_id` **format.** Stable string we can store as text? (Firebase doc ID is fine — we just need it never to change.)
2. **ID stability across script edits.** For pages, scenes/songs, acts, and cues alike — when something is renumbered, renamed, or repositioned, do the IDs survive? This is the single most important question in this proposal. Our sync matches rows by your ID; if IDs churn on every edit, every sync will silently delete and recreate every row, NULL out every cue note's script-context FK (we use `ON DELETE SET NULL`), and lose the work users put into linking notes to script positions. If your IDs aren't currently stable, we can negotiate a content-derived stable ID (e.g. hash of `act + scene + relative_position`) — but a real stable ID is better.
3. **Are you willing to expose `GET /shows/{id}/structure` roughly as proposed in §4.1?** If a different shape suits your data model better, propose it — we can map.
4. **Can you POST a webhook on structure changes?** If not, we'll poll, but webhooks are much nicer for users.
5. **Script viewer URL pattern.** Will you accept `?cue={id}` (or `?page={page_number}`) as a deep-link target?
6. **Cue → scene/song relationship.** Do your cues live *inside* scenes/songs, or just *on pages*? Affects whether `scene_song_id` on cues is required or nullable in the structure payload.
7. **Join key preference.** Would you prefer our `production_id` (UUID) or `short_code` (6-char) as the join key on your side?
8. **Rate-limit expectations.** Any quotas we should design around for the structure endpoint?
9. **Auth scheme.** HMAC per-production as described in §5 — workable on your side, or would something else fit your infra better?
10. **Anything we missed.** Anything about your data model that doesn't map cleanly onto pages / acts / scenes / songs / cues?

---

9. Phased rollout

Phase	LX Notes work	TheCuelist work	User-visible result
1	Linking UI, `cuelist_cues` table, sync route (accepts paste-JSON for early validation)	—	Devs can validate the data mapping end-to-end before TheCuelist API exists.
2	Replace paste with `fetch()` to TheCuelist API. Webhook receiver.	Build `GET /shows/{id}/structure` + outbound `structure-updated` webhook.	Live sync. Notes can be filtered/sorted by script position.
3	Cue autocomplete in Add-Note dialog. Auto-fill page / act / scene / song on note save.	—	Feature #1 and #2 land.
4	”Open in TheCuelist” link on every cue note. URL-param handler on `/notes`.	”Create LX Note” button on each cue in the script viewer.	Features #3 and #4 land. Bidirectional navigation is complete.

Phase 1 unblocks itself — LX Notes can build that whether TheCuelist is ready or not. Everything from Phase 2 onward needs joint work.

---

10. Contract testing

To keep both sides confident their implementations match this spec without coordinating every change through a meeting, we propose three artifacts both teams check into their repos and run in CI:

10.1 JSON SCHEMA FOR THE STRUCTURE RESPONSE

LX Notes will publish a JSON Schema (Draft 2020-12) at a public URL (e.g. `https://www.lxnotes.app/schemas/cuelist-structure.v1.json`). Both sides validate against it:

- **TheCuelist:** in CI, hit `GET /api/v1/shows/{test_show_id}/structure` against your own service and validate the response. Schema mismatch fails the build.

- **LX Notes:** in CI, feed a canonical fixture (committed to our repo) through our sync pipeline and assert no schema errors.

10.2 HMAC TEST VECTOR

A fixed input/output pair that any HMAC implementation can reproduce. Both sides include a test that pins it:

```
Secret:    "test-secret-do-not-use-in-prod"
Timestamp: "1715616000"
Payload:   "/api/v1/shows/abc123/structure"
Expected:
"sha256=4f8c6c5e3b1e7a9d2f0b5c8e1a4d7f0b2e5a8c1d4e7f0a3b6c9d2e5f8a1b4c7d"
```

(Above is illustrative — actual digest computed once and committed.) If your implementation produces a different digest for this input, the signing canonicalization is wrong somewhere. Fail fast.

10.3 REFERENCE FIXTURE: MINIMAL STRUCTURE PAYLOAD

A small but complete `structure.example.json` committed to both repos, used by both sides' tests. Any breaking change to the spec bumps the schema version (`v1` → `v2`) and updates the fixture; we never silently mutate the v1 contract.

---

11. Appendix

11.1 GLOSSARY

Term	Meaning here
Production (LX Notes)	One run of a show (e.g., “Hamilton at Lyric Theatre Sept 2026”). Our top-level scope.
Show (TheCuelist)	Roughly equivalent unit on your side.
Cue	A discrete lighting / sound moment in the script. Has a label like “Cue 47” or “Cue 47.5.”
Cue note	An LX Notes entry tied to a cue. Always has a `cue_number` string; optionally has `script_page_id` and `scene_song_id`.
Scene / Song	A named structural unit of the show. Sits inside an act, spans one or more pages.
Act	The top-level structural unit. Contains scenes/songs.
Page	A page of the script. Identified by `page_number` (often a string — `"42a"` is valid).
Linked production	An LX Notes production paired 1:1 with a TheCuelist show via the linking flow in §6.

11.2 FULL SAMPLE STRUCTURE PAYLOAD

See §4.1 for the canonical shape.

11.3 REFERENCE: LX NOTES' EXISTING SCHEMA WE'LL POPULATE

```
// notes table
{
  cueNumber: string,     // already populated by users
  scriptPageId: string,  // populated by integration
  sceneSongId: string,   // populated by integration
}

// script_pages table (existing + new cuelist_id column)
{ id, page_number, production_id, act_name,
  act_first_cue_number, first_cue_number,
  cuelist_id /* NEW: text, UNIQUE per production */ }

// scenes_songs table (existing + new cuelist_id column)
{ id, name, type, order_index, act_id,
  script_page_id, continues_from_id, module_type, production_id,
  cuelist_id /* NEW: text, UNIQUE per production */ }

// productions table (existing + new cuelist columns)
{ id, name, short_code, /* ... existing ... */
  cuelist_show_id /* NEW: text, UNIQUE */,
  cuelist_api_secret /* NEW: text, encrypted at rest */,
  cuelist_linked_at /* NEW: timestamptz */,
  cuelist_last_sync_at /* NEW: timestamptz */,
  cuelist_last_sync_error /* NEW: text, nullable */ }

// cuelist_cues table (new — cache mirror)
{ id, production_id, cuelist_cue_id, cue_label,
  script_page_id, scene_song_id, order_index, last_synced_at }
```

```
-- New upsert RPCs for the CueList sync path (preserves UUIDs, survives FK refer-
ences):
upsert_script_pages_by_cuelist_id(p_production_id UUID, p_pages JSONB)
upsert_scenes_songs_by_cuelist_id(p_production_id UUID, p_scenes_songs JSONB)
-- Behavior: INSERT … ON CONFLICT (production_id, cuelist_id) DO UPDATE.
-- Rows present in DB but absent from the payload are deleted (transactional).

-- Existing RPCs stay for the manual script-entry admin flow (delete-then-
insert):
replace_script_pages(p_production_id UUID, p_pages JSONB)
replace_scenes_songs(p_production_id UUID, p_scenes_songs JSONB)
```

11.4 CONTACT & TIMELINE

- **LX Notes contact:** Nick (nick@solyomdesign.com)

- **Proposed cadence:** one short call to walk through this doc, then async on the open questions.
- **No deadline** — we're happy to move at whatever pace works for your team.

Document version: 1.0 — 2026-05-13